COMPUTER ENGINEERING

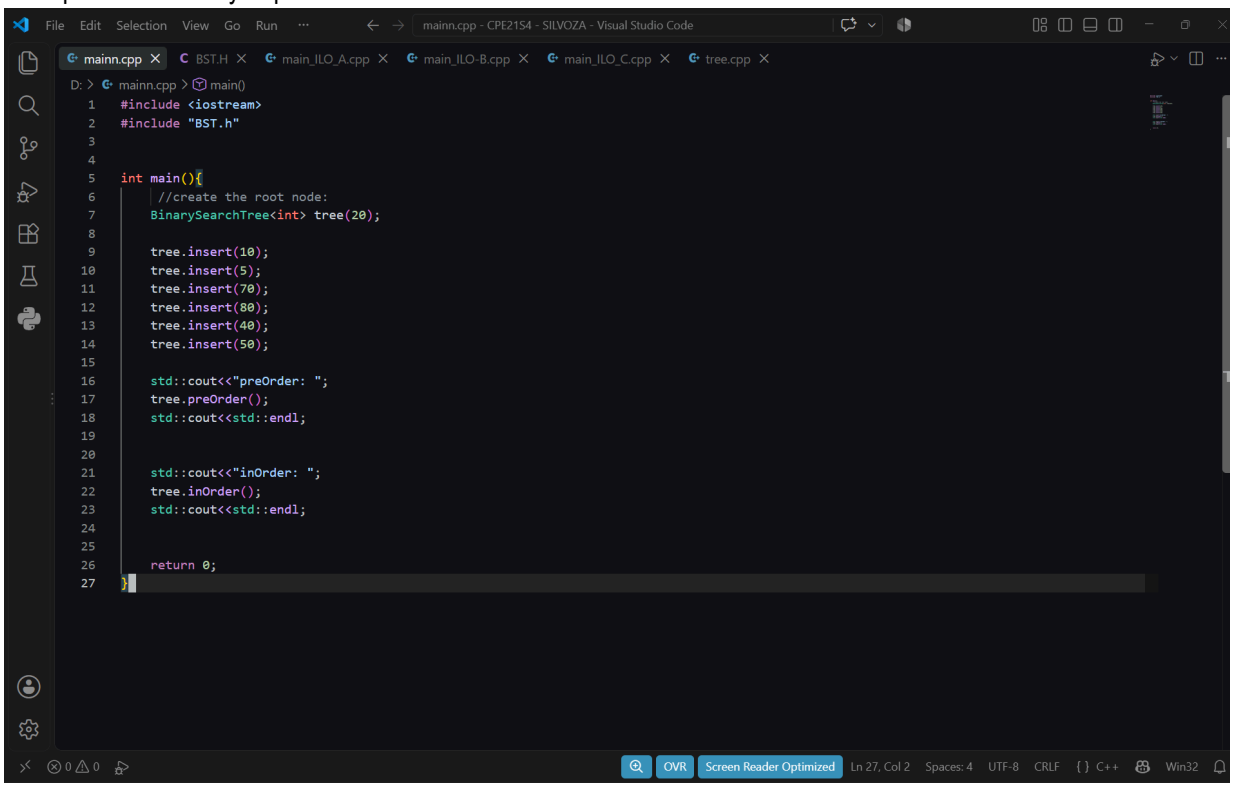
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	09/23/2026	**DATE SUBMITTED:**	09/23/2026
NAME:	KEITH NICOLE SILVOZA	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. 9.1 TREE ADT

1. PROCEDURE OUTPUT

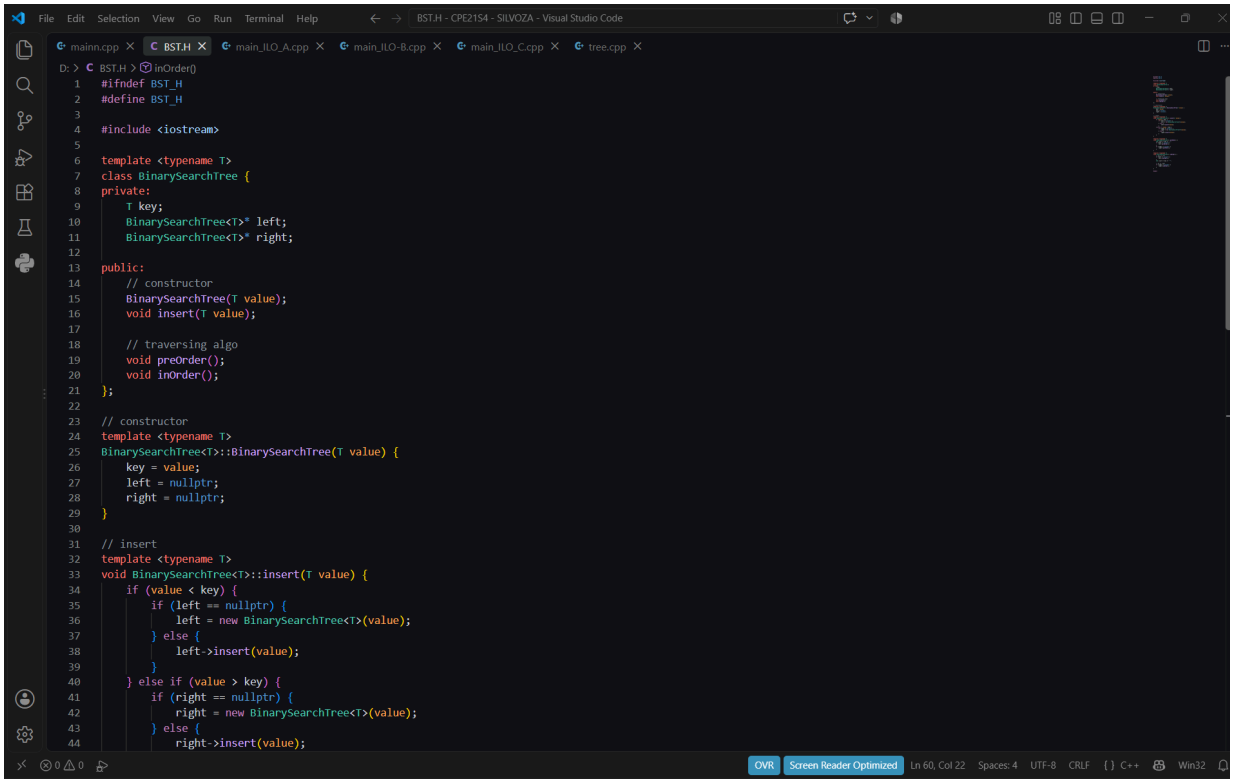
What procedures did you perform?




```
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>  ${env:DEBUGINFOD_URLS}
s\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-h24wak5h.c5t' '--stdout=Microsoft
gine-Pid-3eo4nnbv.on2' '--dbgExe=D:\mingw64\bin\gdb.exe' '--interpreter=mi'
preOrder: 20 10 5 70 40 50 80
inOrder: 5 10 20 40 50 70 80
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA> (Set-ExecutionPolicy -S
ts\CPE21S4 - SILVOZA\.venv\Scripts\Activate.ps1")
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>
```

COMPUTER ENGINEERING

QUEZON CITY



```cpp
#ifndef BST_H
#define BST_H

#include <iostream>

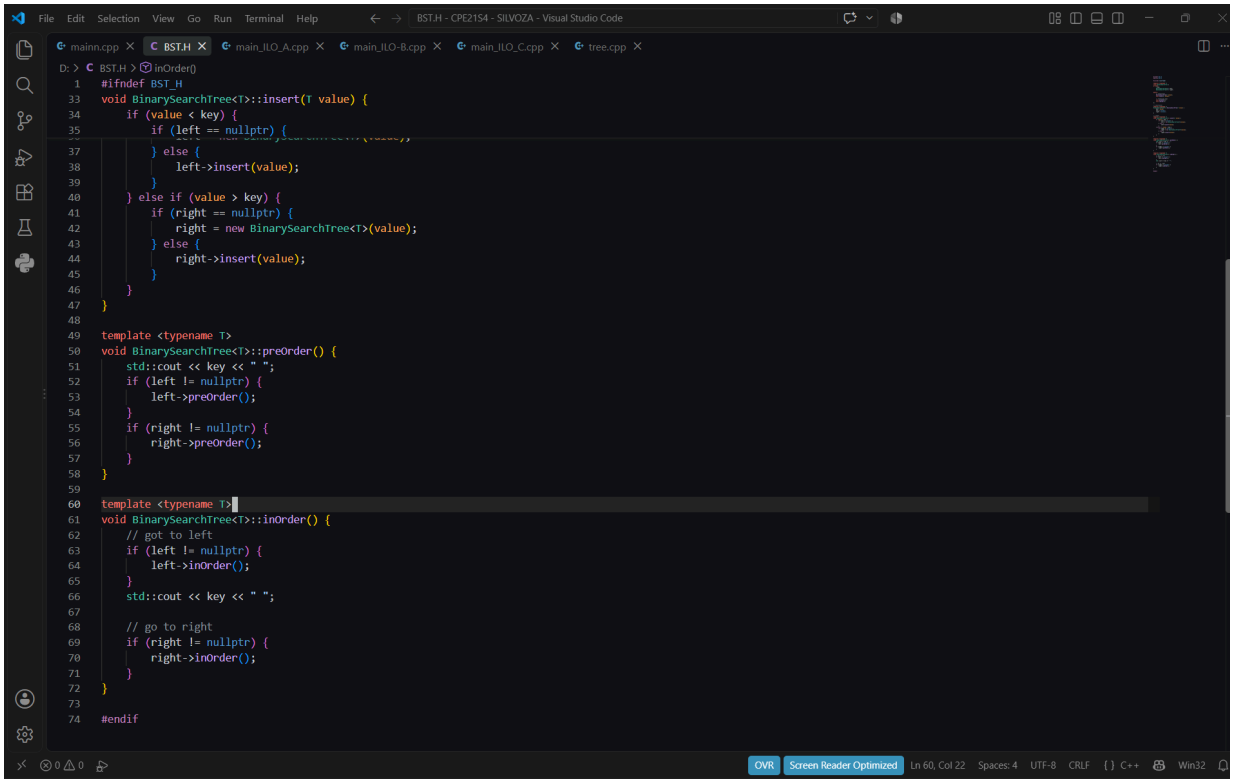
template <typename T>
class BinarySearchTree {
private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

public:
    // constructor
    BinarySearchTree(T value);
    void insert(T value);

    // traversing algo
    void preOrder();
    void inOrder();
};

// constructor
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    key = value;
    left = nullptr;
    right = nullptr;
}

// insert
template <typename T>
void BinarySearchTree<T>::insert(T value) {
    if (value < key) {
        if (left == nullptr) {
            left = new BinarySearchTree<T>(value);
        } else {
            left->insert(value);
        }
    } else if (value > key) {
        if (right == nullptr) {
            right = new BinarySearchTree<T>(value);
        } else {
            right->insert(value);
```



```cpp
#ifndef BST_H
void BinarySearchTree<T>::insert(T value) {
    if (value < key) {
        if (left == nullptr) {
        } else {
            left->insert(value);
        }
    } else if (value > key) {
        if (right == nullptr) {
            right = new BinarySearchTree<T>(value);
        } else {
            right->insert(value);
        }
    }
}

template <typename T>
void BinarySearchTree<T>::preOrder() {
    std::cout << key << " ";
    if (left != nullptr) {
        left->preOrder();
    }
    if (right != nullptr) {
        right->preOrder();
    }
}

template <typename T>
void BinarySearchTree<T>::inOrder() {
    // got to left
    if (left != nullptr) {
        left->inOrder();
    }
    std::cout << key << " ";

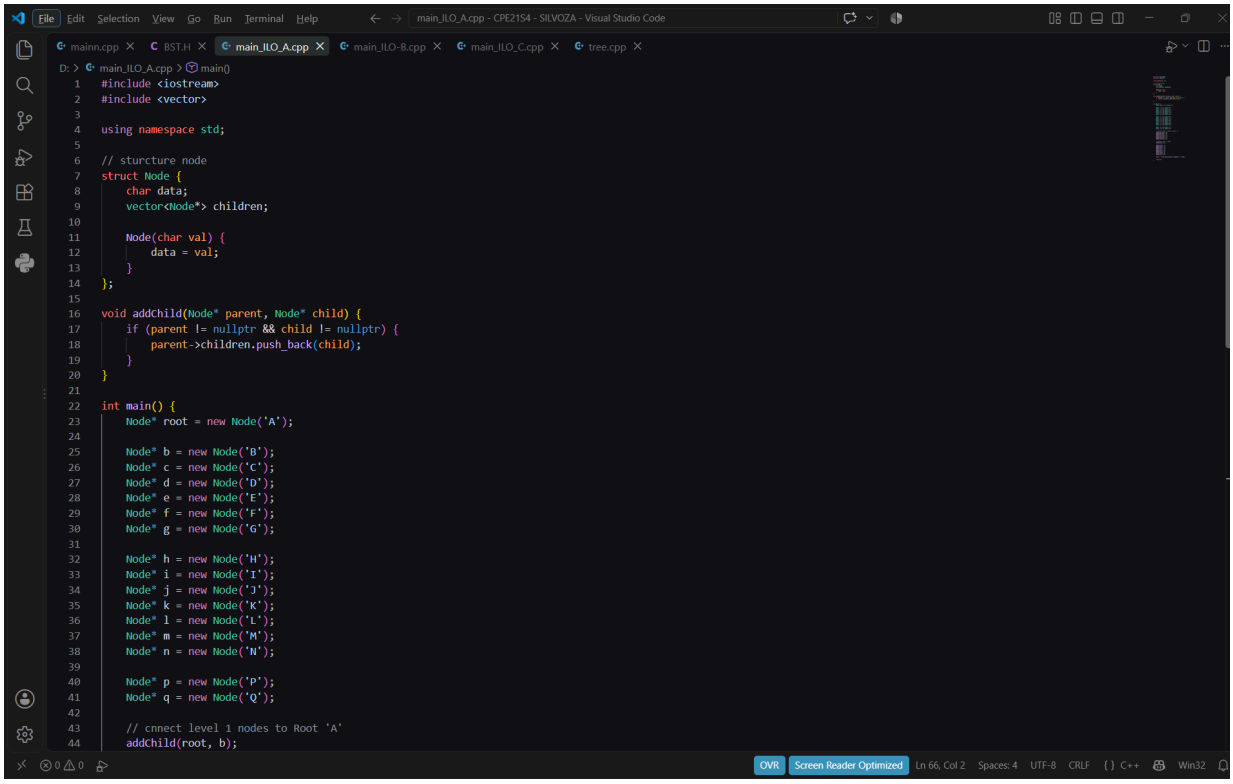
    // go to right
    if (right != nullptr) {
        right->inOrder();
    }
}

#endif
```

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY



```cpp
#include <iostream>
#include <vector>

using namespace std;

// stucture node
struct Node {
    char data;
    vector<Node*> children;

    Node(char val) {
        data = val;
    }
};

void addChild(Node* parent, Node* child) {
    if (parent != nullptr && child != nullptr) {
        parent->children.push_back(child);
    }
}

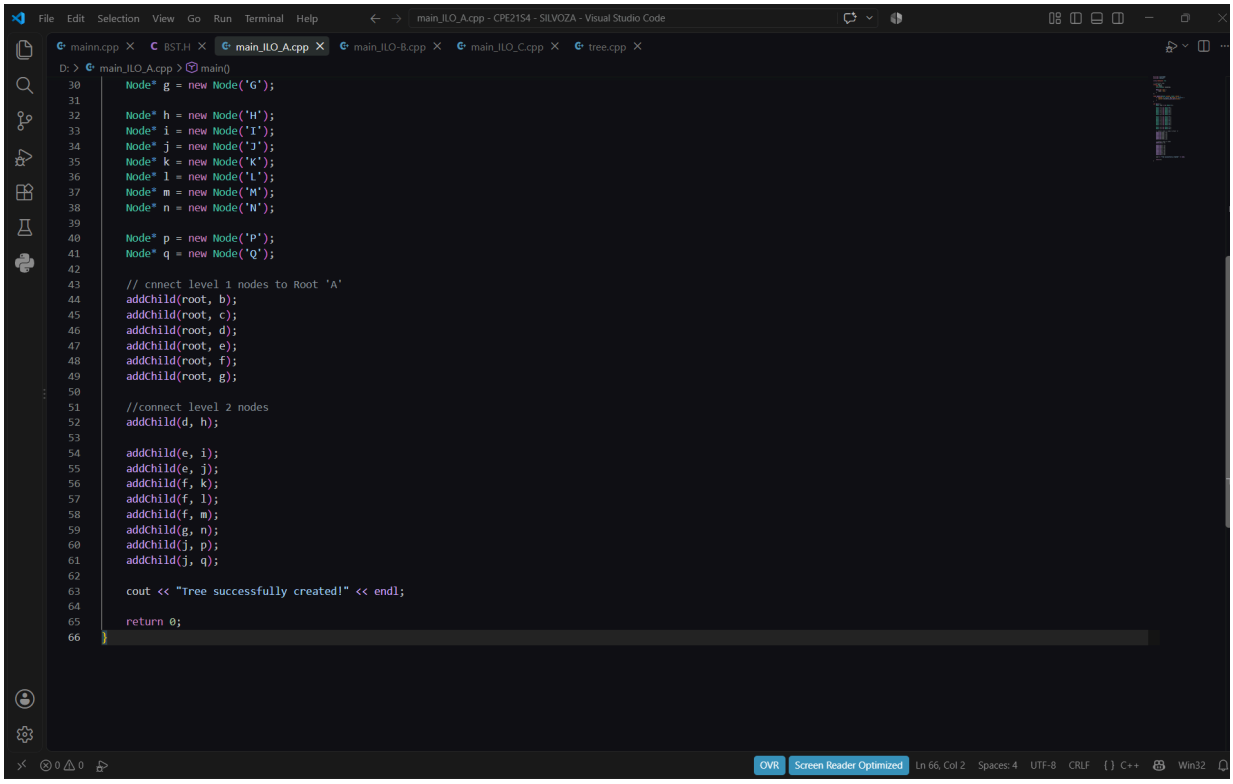
int main() {
    Node* root = new Node('A');

    Node* b = new Node('B');
    Node* c = new Node('C');
    Node* d = new Node('D');
    Node* e = new Node('E');
    Node* f = new Node('F');
    Node* g = new Node('G');

    Node* h = new Node('H');
    Node* i = new Node('I');
    Node* j = new Node('J');
    Node* k = new Node('K');
    Node* l = new Node('L');
    Node* m = new Node('M');
    Node* n = new Node('N');

    Node* p = new Node('P');
    Node* q = new Node('Q');

    // cnnect level 1 nodes to Root 'A'
    addChild(root, b);
```



```cpp
    Node* g = new Node('G');

    Node* h = new Node('H');
    Node* i = new Node('I');
    Node* j = new Node('J');
    Node* k = new Node('K');
    Node* l = new Node('L');
    Node* m = new Node('M');
    Node* n = new Node('N');

    Node* p = new Node('P');
    Node* q = new Node('Q');

    // cnnect level 1 nodes to Root 'A'
    addChild(root, b);
    addChild(root, c);
    addChild(root, d);
    addChild(root, e);
    addChild(root, f);
    addChild(root, g);

    //connect level 2 nodes
    addChild(d, h);

    addChild(e, i);
    addChild(e, j);
    addChild(f, k);
    addChild(f, l);
    addChild(f, m);
    addChild(g, n);
    addChild(j, p);
    addChild(j, q);

    cout << "Tree successfully created!" << endl;

    return 0;
}
```

COMPUTER ENGINEERING

QUEZON CITY

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>  ${env:DEBUGINFOD_URL
s\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-agvfkjye.yke' '--stdout=Microso
gine-Pid-bjprbxu2.j1j' '--dbgExe=D:\mingw64\bin\gdb.exe' '--interpreter=mi'
Tree successfully created!
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA> (Set-ExecutionPolicy
ts\CPE21S4 - SILVOZA\.venv\Scripts\Activate.ps1")
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>
```

```cpp
#include <iostream>
#include <vector>

using namespace std;

struct Node {
    char data;
    vector<Node*> children;

    Node(char val) {
        data = val;
    }
};

void addChild(Node* parent, Node* child) {
    if (parent && child) {
        parent->children.push_back(child);
    }
}

// pre order
void preOrder(Node* root) {
    if (!root) return;
    cout << root->data << " ";
    for (Node* child : root->children) {
        preOrder(child);
    }
}

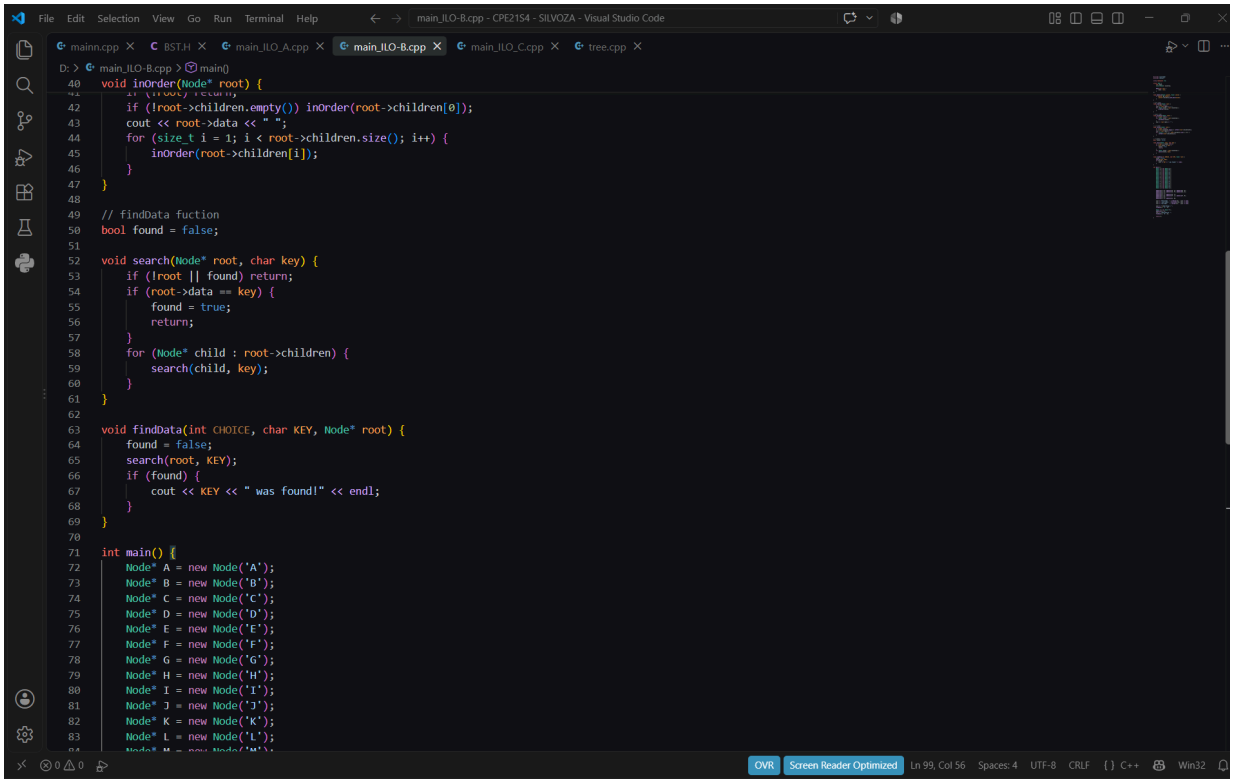
// post order
void postOrder(Node* root) {
    if (!root) return;
    for (Node* child : root->children) {
        postOrder(child);
    }
    cout << root->data << " ";
}

// in order
void inOrder(Node* root) {
    if (!root) return;
    if (!root->children.empty()) inOrder(root->children[0]);
    cout << root->data << " ";
    for (size_t i = 1; i < root->children.size(); i++) {
```

T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY



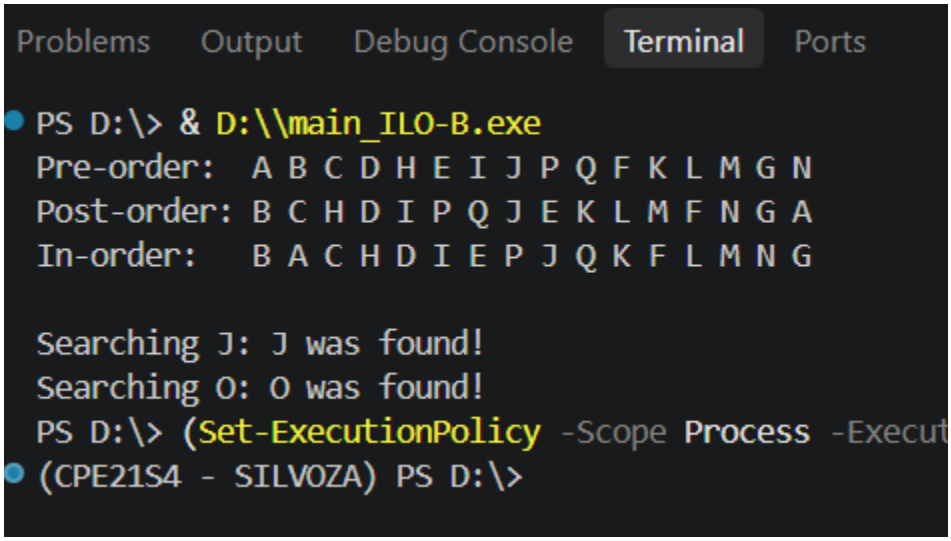
```cpp
void inOrder(Node* root) {
    if (!root->children.empty()) inOrder(root->children[0]);
    cout << root->data << " ";
    for (size_t i = 1; i < root->children.size(); i++) {
        inOrder(root->children[i]);
    }
}

// findData fuction
bool found = false;

void search(Node* root, char key) {
    if (!root || found) return;
    if (root->data == key) {
        found = true;
        return;
    }
    for (Node* child : root->children) {
        search(child, key);
    }
}

void findData(int CHOICE, char KEY, Node* root) {
    found = false;
    search(root, KEY);
    if (found) {
        cout << KEY << " was found!" << endl;
    }
}

int main() {
    Node* A = new Node('A');
    Node* B = new Node('B');
    Node* C = new Node('C');
    Node* D = new Node('D');
    Node* E = new Node('E');
    Node* F = new Node('F');
    Node* G = new Node('G');
    Node* H = new Node('H');
    Node* I = new Node('I');
    Node* J = new Node('J');
    Node* K = new Node('K');
    Node* L = new Node('L');
```



```
Problems  Output  Debug Console  Terminal  Ports

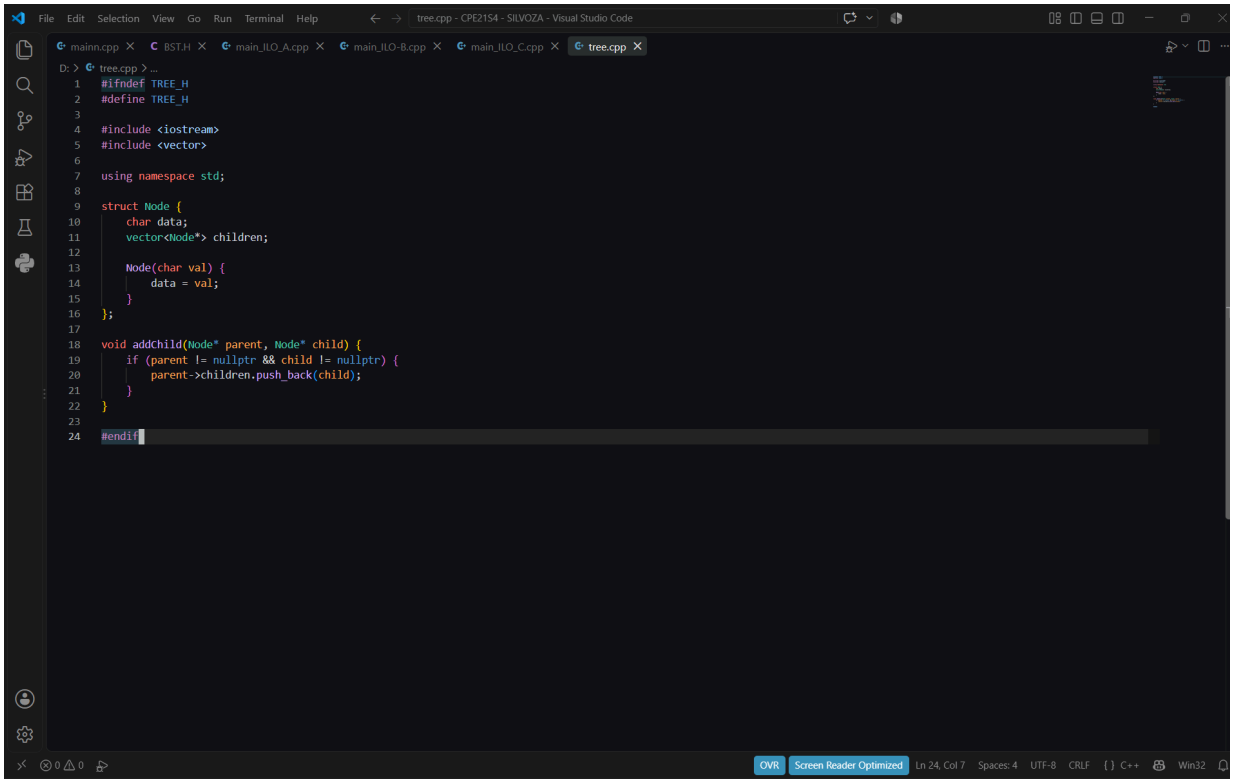
PS D:\> & D:\\main_ILO-B.exe
Pre-order:  A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A
In-order:   B A C H D I E P J Q K F L M N G

Searching J: J was found!
Searching O: O was found!
PS D:\> (Set-ExecutionPolicy -Scope Process -Execut
(CPE21S4 - SILVOZA) PS D:\>
```

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

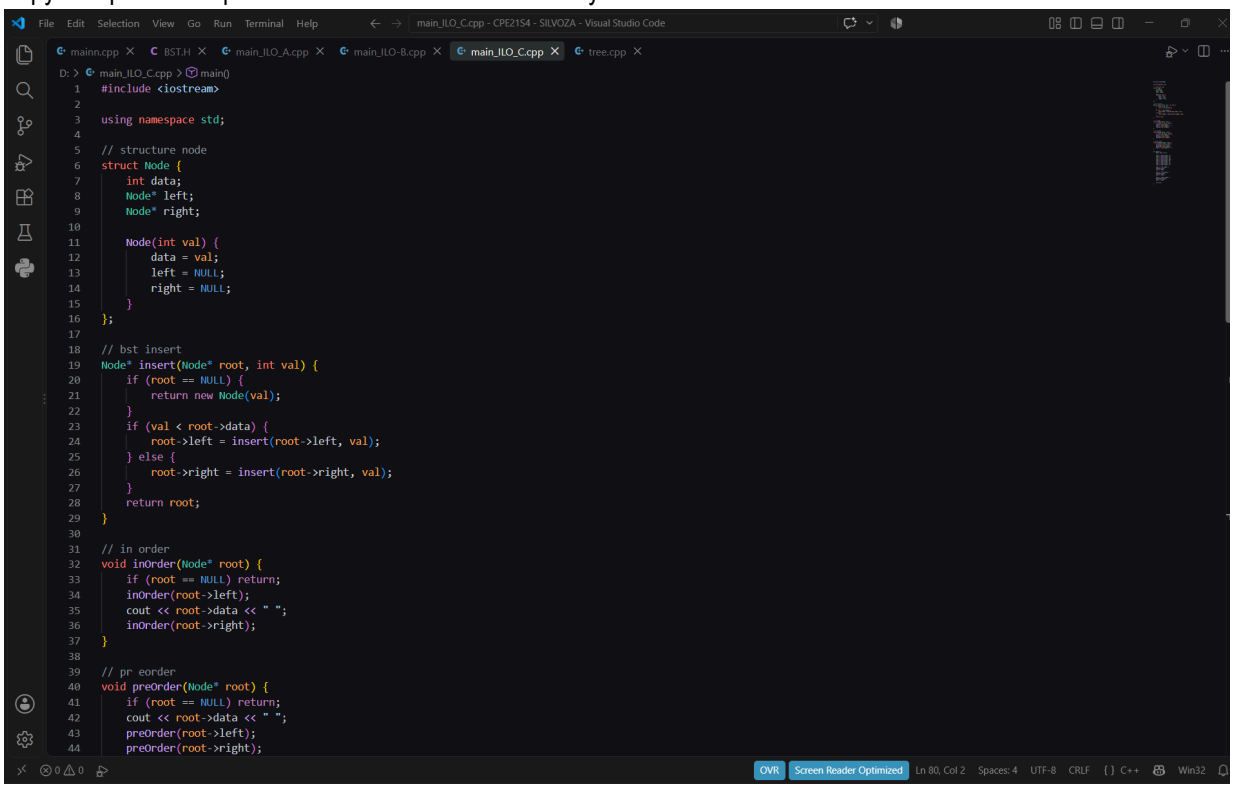


COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

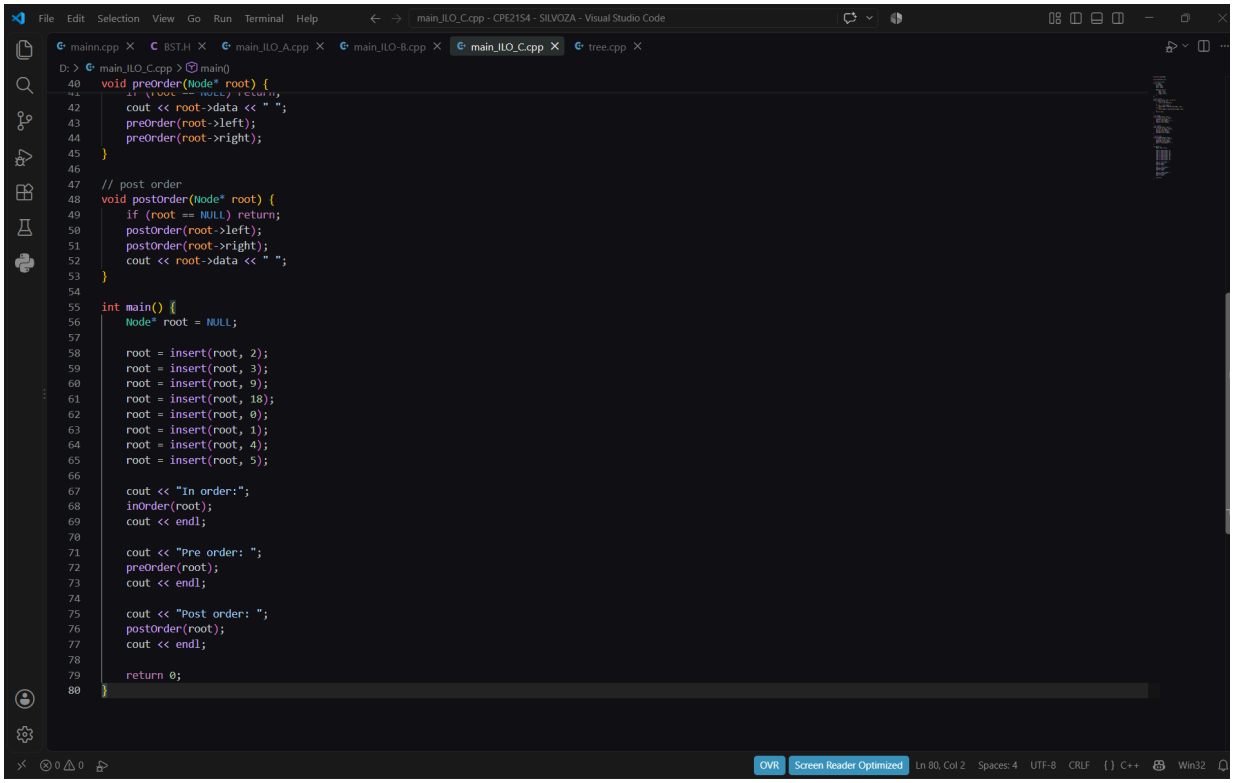
Copy and paste the questions from the lab and include your answers after.



T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY



```cpp
void preOrder(Node* root) {
    if (root == NULL) return;
    cout << root->data << " ";
    preOrder(root->left);
    preOrder(root->right);
}

// post order
void postOrder(Node* root) {
    if (root == NULL) return;
    postOrder(root->left);
    postOrder(root->right);
    cout << root->data << " ";
}

int main() {
    Node* root = NULL;

    root = insert(root, 2);
    root = insert(root, 3);
    root = insert(root, 9);
    root = insert(root, 18);
    root = insert(root, 0);
    root = insert(root, 1);
    root = insert(root, 4);
    root = insert(root, 5);

    cout << "In order:";
    inOrder(root);
    cout << endl;

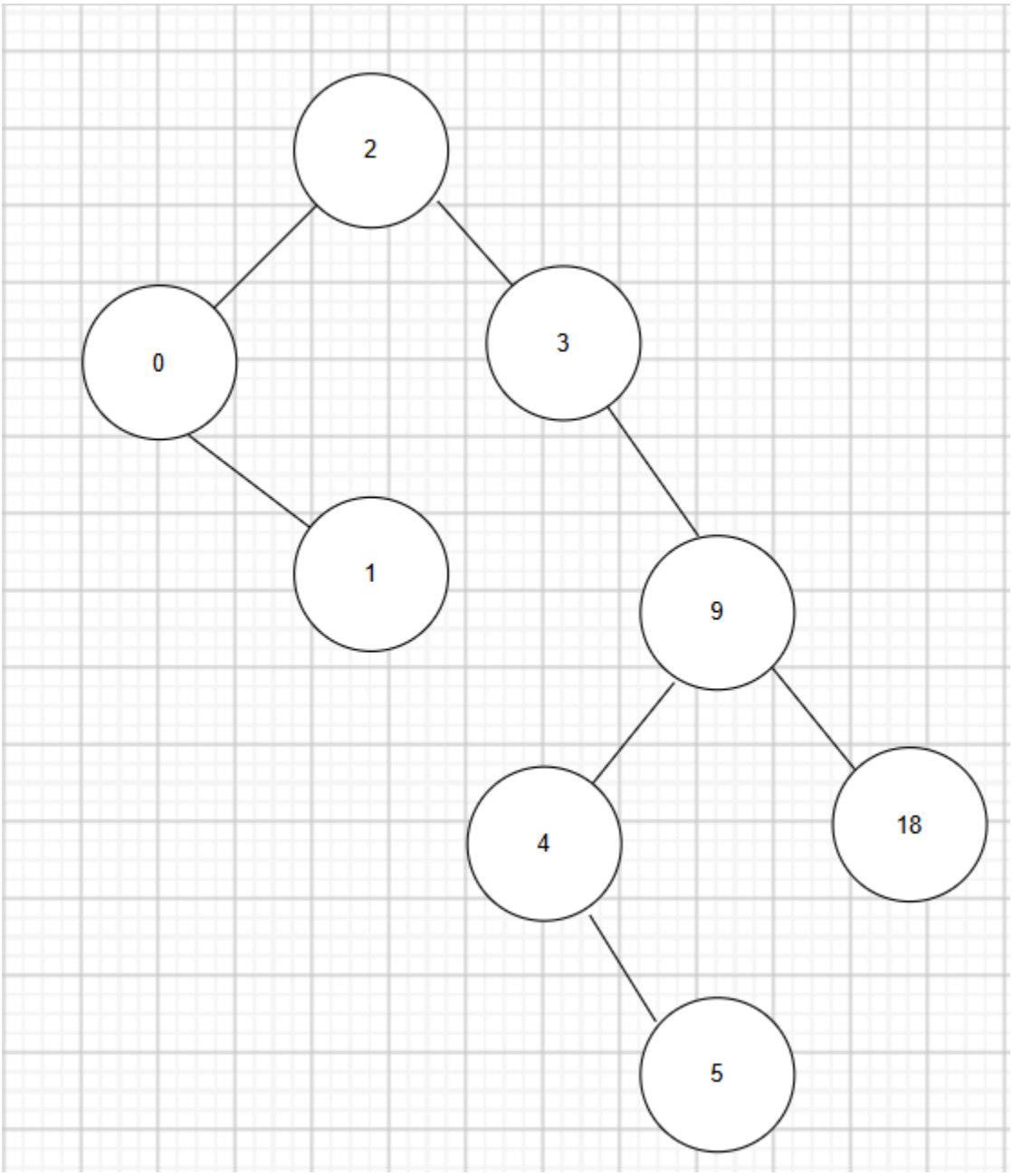
    cout << "Pre order: ";
    preOrder(root);
    cout << endl;

    cout << "Post order: ";
    postOrder(root);
    cout << endl;

    return 0;
}
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>  ${env:DEBUGINFOD_URLS
s\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-in35ospr.krf' '--stdout=Microsof
gine-Pid-bqlm35kq.2ok' '--dbgExe=D:\mingw64\bin\gdb.exe' '--interpreter=mi'
In order:0 1 2 3 4 5 9 18
Pre order: 2 0 1 3 9 4 5 18
Post order: 1 0 5 4 18 9 3 2
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA> (Set-ExecutionPolicy -
ts\CPE21S4 - SILVOZA\.venv\Scripts\Activate.ps1")
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>
```



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance and explanation of concepts**

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to assist in debugging syntax errors and explaining tree data structure traversal concepts. All code implementation, analysis, and required work were completed and verified independently without replacing original thinking or learning.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.